

异常大额伤害过滤器 · 完整设计规范

本文档定位：v6.7 / v7 阶段引入的"异常大额伤害平滑过滤器"完整设计。在现有 `L9_FILTER` (v6.5.2 固定值黑名单) + `suspectVictims` (v6.5.8 可疑怪物名单) + `L8 carry 剥离` (v6.5.9) 的基础上，把过滤体系从"硬编码黑名单"演进为"基于上下文的多信号判定 + 自适应 P95 Outlier 检测"。

关联文档：

- `V6_STATS_DEV_PLAN.md` §阶段② · 副产品 —— 死亡锚点过滤器初版设想（本文档为正式定稿）
- `V6_STATS_STATUS.md` —— v6 设计 vs 实现 gap 分析
- `MAP_DATAPACK_REFERENCE.md` §6.3 / §11.3 —— `Defence` / `*Armor` / `Damage` scoreboard 定义
- `CLIENT_SIDE_STATS_PROPOSAL.md` §4.3 —— 双轨数据源策略（粒子 vs 心数 delta），本过滤器需双端对称

历史教训锚点：

- v6.3.8 全局 `HARD_CAP` → 高阶 BOSS 真伤爆表，回滚
- v6.3.10 均值×K=5 → 误伤玩家高光致死单击，回滚
- 本设计的核心改进：**致死守门 R1 + P95 替代均值 + 武器/关卡切换 reset + 死亡锚点缓冲** 四道保险串联，让 K 可以放心拉到 50

目录

- 1. [设计原则（铁律）](#)
- 2. [异常源分类学](#)
- 3. [决策树总流程](#)
- 4. [各层细节](#)
- 5. [P95 计算规范](#)
- 6. [配置项清单](#)
- 7. [击杀人头与 contributors 整合](#)
- 8. [L9_FILTER reason 子标签扩展](#)
- 9. [验收用例](#)
- 10. [分阶段落地](#)
- 11. [与失败教训的对照](#)
- 12. [客户端等价 \(v7 fallback\)](#)

1. 设计原则（铁律）

#	铁律	含义	反例
R1	零误伤致死高光单击	致死单击若数值合理，无条件放行（既计伤害，又计击杀）	v6.3.10 均值×K 拦掉 BOSS 收尾 9999

#	铁律	含义	反例
R2	致死分两类	致死单击 + 数值合理 = 玩家高光（全计入）；致死单击 + 数值异常 = 机制斩杀（仅计击杀人头，不计伤害）	把所有致死无脑放行 → 机制刀污染玩家伤害账户
R3	不依赖单一信号	每条规则给出 <code>reason</code> + 多信号合议判定，避免一刀切	v6.3.8 单看 hard cap
R4	过滤即诊断可见	被过滤的事件 不进玩家账户 ，但 进 L9_FILTER 桶 + 聊天栏可见 ，附 reason 标签	v6.5.1 直接 return 把回血粒子吞了
R5	状态/世界先于数值	先用 victim 状态 / 地图阶段 flag 判定，再退到数值规则	仅靠 damage 大小判定
R6	配置可关、参数可调	每条规则独立开关，避免一刀切回滚	v6.3.10 只能整体回滚
R7	服务端 / 客户端规则集对称	同一异常源在双端出同样判定（归属精度差异可接受）	双端各搞一套语义不同的过滤

2. 异常源分类学

把"异常大额伤害"按**根因**分成 7 类，每类有不同特征与处置：

类	根因	触发方式	数值特征	现状处置	本设计处置
A	怪物初始化 / 复活 set	<code>scoreboard players set @s RedHearts <N></code>	固定离散值 (1000 / 9000 / 10000 / 100000)	✅ <code>isInitHpJump</code> 黑名单	沿用 G3
B	形态切换 / 阶段 set	同上但 N 浮动	浮动但 ≥ 当前阶段日常 P95	✅ v6.5.8 可疑怪物名单（幽匿骷髅/幽匿僵尸 ≥800）	沿用 G4
C	批量结算 / <code>kill @e</code>	单 tick 内多 victim 4 层心数全清	极大值（接近 victim MaxHP）	❌ 漏	G5 mass-wipe 守卫
D	状态机边界 (PauseGame / GameOver / 切关 / GameID 跳)	状态切换 1 tick 内的负 delta	任意值	❌ 漏	G2 状态机守卫
E	单帧多倍 modifier 溢出 (Tier × GlassCannon × Element RNG)	玩家真实暴击 + 元素叠加	极大但 致死	✅ 应保留	R1 致死守门 + P95×K=50 自然放行
F	数据二次记账（地图 BUG）	<code>damage.mcfuction</code> 同 tick 二次执行	与上一帧完全相同	❌ 漏	G6 重放守卫

类	根因	触发方式	数值特征	现状处置	本设计处置
G	DoT 高频小伤害噪声	持续燃烧 / 中毒 / 出血环境效果	数值 < 5、每 tick 都有	✗ 漏 (污染 P95)	G_low 硬地板 5

新增类 G： DoT 高频小伤害不是"大额"异常，但会污染 P95 训练样本（拉低分位数 → 让阈值过紧 → 误伤普通伤害）。所以也归到过滤体系里。

3. 决策树总流程

每条新出现的伤害事件按下表顺序过判定。**任一层判定为 FILTER 立即短路**；全部未命中走原归属链。

```
入口: (victim, damage, tick, source ∈ {DamageShower, RedHearts, *DMG})
|
├─ G0  filterEnabled = false?                是 → 直通归属链（总网关）
|
├─ G_low damage < 5?                          是 → L9_FILTER (low-noise)
|                                             · 不入 P95 · 不进玩家账户
|
├─ G3  isInitHpJump(damage)?                  是 → L9_FILTER (init-hp-jump)
├─ G4  isSuspectVictim(victim, damage)?       是 → L9_FILTER (suspect-victim)
├─ G2  状态机边界
|      (#PauseGame > 0 ∨ GameID 跳 ≤ 5t ∨
|      victim 含 Coffin/Prop/NPC tag)?        是 → L9_FILTER (state-boundary)
|
├─ G7a damage > victim.MaxHP × 3 ?            是 → 物理地板触发:
|      └─ 同时致死?                          是 → L9_FILTER (lethal-mechanism)
|                                             · 击杀仍计 contributors
|      └─ 未致死                            否 → L9_FILTER (oversize)
|
├─ G6  重放: 同 victim 同 damage 上一 tick 已记? 是 → L9_FILTER (duplicate)
|
└─ 进 PendingDamageBuffer, 3 tick 后 resolve:
    └─ victim 已死 + damage > P95 × 50      → L9_FILTER (lethal-mechanism)
        |                                     · 击杀仍计 contributors
        └─ victim 已死 + damage ≤ P95 × 50   → 玩家高光 · 完整放行
        |                                     · 不入 P95 训练 (致死不入)
        └─ victim 仍活 + damage > P95 × 50   → L9_FILTER (outlier)
        └─ G5 mass-wipe 整批检查命中         → 整批回收 L9_FILTER (mass-wipe)
        └─ 其它                             → 放行 + 入 P95 训练样本
```

G5 为什么放在 buffer 末尾： mass-wipe 需要"同 tick ≥ 3 个 victim 都满足条件 1 (damage / MaxHP ≥ 0.95) 且归属各异"——必须等本 tick 的所有事件都 buffer 完才能整批检查。详见 §4.6。

4. 各层细节

4.1 G0 · 全局开关

`ServerConfig.filterEnabled` (默认 `true`)

`false` 时所有过滤规则跳过，相当于关掉本子系统。用于 dev 环境快速对比"开/关过滤"的差异。

4.2 G_low · 低伤害噪声硬地板

`ServerConfig.lowDamageFloor` (默认 5)

`ServerConfig.defenceExclusionThreshold` (默认 50)

判定规则：

```
若 damage < lowDamageFloor:
  且 readVictimDefence(victim) ≤ defenceExclusionThreshold:    // ← v6.7.1 新增豁免
    → forceLayer = L9_FILTER
    → reason = "low-noise"
    → 不入 P95 训练样本
    → 不写 PlayerDamageStats
    → 不写 sessionTotal / maxHit
  否则（高护甲怪）：
    → 跳过 G_low，继续后续规则；伤害正常进玩家账户
```

鲁棒性论证：

- CTT 武器单击最低也在 5+（赤手都 ≥ 1 但通过 G_low 过滤后赤手不再被误算"参战"）
- 石头僵尸 / 远古卫士这类高 Defence 怪的 1~4 点伤害是合法物理伤害的减伤后输出——v6.7.1 加 Defence 豁免把它们排除在 G_low 之外，伤害仍计入玩家账户（与 §5.1 P95 入样规则中的"Defence > 50 不入样但伤害仍记账"一脉相承）
- 偶然 1~4 点的合法伤害如果**致死了 victim** → R1 守门兜底放行（G_low 在 R1 之前？看 §3 顺序：**G_low 在最前**，所以 1 点致死也会被过滤）
- **取舍**：宁可漏 1 点的最后一刀，也不愿意被 DoT 高频污染 P95
- 玩家可在 ConfigScreen 把 `lowDamageFloor` 调到 0 关闭本规则

DoT 致死的处理：火焰 DoT 1 点烧死小怪——这一刀进 L9_FILTER (low-noise)，但**击杀仍由**

`VictimDamageContributors` **兜底**（点火者已经在 contributors 里）。如果 contributors 是空的（怪是被环境 DoT 烧死，没人点过火），进未分类击杀。**与机制斩杀同一逻辑。**

Defence 豁免的实现细节：地图无 Defence scoreboard 时 `getNullableObjective` 返回 null → `readVictimDefence` 返回 0 → $0 \leq 50$ → 不豁免，G_low 正常工作。降级行为安全。

4.3 G3 · 固定值黑名单（沿用 v6.5.2）

```
ServerConfig.filterInitHpJumps (默认 true)
ServerConfig.initHpJumpValues (默认 {1000, 9000, 10000, 100000})
```

`isInitHpJump(damage)` 命中 → `forceLayer = L9_FILTER`, `reason = "init-hp-jump"`。

不动——v6.5.2 已稳定。

4.4 G4 · 可疑怪物名单（沿用 v6.5.8）

```
ServerConfig.filterSuspectVictims (默认 true)
ServerConfig.suspectVictims (默认 {"幽匿骷髅", "幽匿僵尸"})
ServerConfig.suspectVictimDamageThreshold (默认 800)
```

`isSuspectVictim(victim, damage)` 命中 → `forceLayer = L9_FILTER`, `reason = "suspect-victim"`。

不动——v6.5.8 已稳定。

4.5 G2 · 状态机边界守卫（新）

新增配置：

```
ServerConfig.filterStateBoundary (默认 true)
ServerConfig.sessionBoundaryGuardTicks (默认 5)
ServerConfig.suspectVictimTags (默认 {"Coffin", "Prop", "NPC", "TestDummy", "Debug"})
```

判定（任一条件命中即过滤）：

条件	reason
<code>#PauseGame CTT > 0</code>	<code>paused</code>
<code>tick - StageBoundaryDispatcher.lastGameIdChangeTick() < 5</code>	<code>session-boundary</code>
<code>victim commandTags</code> 含上面任一 tag	<code>suspect-tag</code>

为什么必要：

- `#PauseGame` 期间地图脚本可能调整怪物状态产生假 delta
- GameID 跳变（局结束 / 新局开始）那 1~5 tick 内的 score 变动是清场而非战斗
- `Coffin` 是地图特有的"假尸体"实体，`RedHearts ≤ 0` 但不死，无意义伤害源
- `Prop` / `TestDummy` / `NPC` / `Debug` 不该有伤害事件

4.6 G7a · 物理地板（新，最锋利）

新增配置：

```
ServerConfig.physicalCeilMultiplier (默认 3)
```

判定规则：

```
读 victim 的 MaxHP scoreboard
若 MaxHP > 0 且 damage > MaxHP × 3:
  若 致死 (victim.RedHearts ≤ 0 或 victim.isRemoved()):
    → forceLayer = L9_FILTER
    → reason = "lethal-mechanism"
    → 击杀照样写 contributors (详见 §7)
  否则:
    → forceLayer = L9_FILTER
    → reason = "oversize"
```

这条规则是最锋利的——

- 不需要 P95 样本（首战即生效）
- 物理意义清晰：CTT 怪物 4 层心数 + 各种吸收，单击打掉 3 倍 MaxHP 已经超出任何合理设计
- 用户案例：小吃货 MaxHP ≈ 500 → 阈值 1500，本次 103950 远超 → 立即 FILTER

边界：

场景	MaxHP	阈值	致死时处置	非致死时处置
小吃货被脚本 set RedHearts=-103950	500	1500	lethal-mechanism（击杀算给玩家）	oversize（已不太可能：致死大概率发生）
大 BOSS 真实暴击	30000	90000	不会触发 G7a，进 G7b 走 P95 判定	同上
大 BOSS 收尾刀 35000	30000	90000	不触发 G7a → buffer → P95×50 判定（远低于阈值）→ 放行	—

4.7 G6 · 重放守卫（新，便宜）

```
ServerConfig.filterDuplicateReplay (默认 true)
```

PendingDamageBuffer 里若已存在 (victim, damage, tick - 1) 完全相同的记录 → 当前事件 → L9_FILTER (duplicate)。

预期命中率 < 0.1%，但实现成本几乎为 0，作 sanity check。

4.8 G7b · 死亡锚点缓冲 + P95 Outlier (核心创新)

4.8.1 缓冲流程

致死判定方式 (简单版, Q3 决议) : 事件入 buffer 时立即读一次 `victim.RedHearts`, 若 `RedHearts ≤ 0` 则记 `entry.wasLethalAtBuffer = true`。3 tick 后 flush 直接读这个 flag, **即使期间被 IronHeart 救活 / 复活也不退判**——伤害仍按 lethal-mechanism 处理 (数值不计入玩家账户), 击杀维度由 contributors 链路独立处理 (怪没真死则没击杀触发, contributors 已写入留待下次真死时归属)。

好处: 不依赖 `victimTombstone` 死亡 race, flush 时只是一次 boolean 读取。

```
PendingDamageBuffer 接收所有"未被 G_low / G2 / G3 / G4 / G7a / G6 短路"的事件

入 buffer 时记下:
entry = {
    victim, damage, attackerHint, bufferedAtTick,
    wasLethalAtBuffer = (victim.RedHearts ≤ 0)           // ← Q3 决议: 入 buffer 那一帧立即定格
}

每 tick 末 (END_SERVER_TICK) 调用 flush(currentTick):
for entry in queue:
    若 currentTick - entry.bufferedAtTick < 3:
        continue // 还没等够, 跳过

出队, 做最终判定 →
├ entry.wasLethalAtBuffer == true (入 buffer 那帧 RedHearts ≤ 0):
│   ├── damage > P95 × 50 → L9_FILTER (lethal-mechanism), contributors 仍写
│   └── damage ≤ P95 × 50 → 玩家高光放行 + 不入 P95 训练样本
│
├ entry.wasLethalAtBuffer == false:
│   └── samples ≥ 20 且 damage > P95 × 50 → L9_FILTER (outlier), contributors 不写
│
└ 其它 (含 P95 未启用 / 数值合理):
    → 放行 + 入 P95 训练样本 (damage ≥ lowDamageFloor 且 victim.Defence ≤ 50)
```

简单版的代价: 怪在 buffer 期间被复活的极少数情况下, 这一刀仍按致死处理, 伤害不计、击杀通过 contributors 后续真死时才结算。判定逻辑简单且无 race, 不依赖 `victimTombstone.diedBetween` 的精确性。

4.8.2 关键参数

```
ServerConfig.deathAnchorGraceTicks (默认 3)
ServerConfig.deathAnchorOutlierMultiplier (默认 50) // 即 K
ServerConfig.deathAnchorMinSamples (默认 20)
```

参数	默认值	调参依据
graceTicks	3	<code>victimTombstone</code> 死亡判定可能慢 1~2 tick; 3 tick 给一帧余量。延迟 = 150 ms 肉眼无感

参数	默认值	调参依据
outlierMultiplier (K)	50	远高于 v6.3.10 失败的 K=5。配合 R1 守门 + 物理地板， K=50 在数学上几乎不可能误伤合法伤害
minSamples	20	武器/关卡切换后的前 ~5 秒纯靠物理地板兜底；样本积累达到 20 后 P95 才启用

4.8.3 P95 启用前的 fallback

样本不足 `minSamples` 时不启用 P95，但**保留上一窗口的 $P95 \times 1.5$ 作为软兜底**：

```
ServerConfig.fallbackToPreviousP95 (默认 true)
ServerConfig.fallbackP95Multiplier (默认 1.5)

samples = currentwindow.size()
若 samples < 20:
  若 fallbackToPreviousP95 且 lastFinalP95 > 0:
    threshold = lastFinalP95 × 1.5 × K
  否则:
    threshold = 不启用 P95 (仅物理地板兜底)
否则:
  threshold = currentP95 × K
```

含义：上一关刚收摊的 P95 = 200，切关后新关样本 < 20 时 → 阈值 = $200 \times 1.5 \times 50 = 15000$ 。这比"完全不启用 P95"严格，比"用旧 P95 直接当新 P95"宽松 ($\times 1.5$ 防止跨关难度差异误伤)。

`lastFinalP95` 在每次 reset (武器切换 / 关卡切换) **前**保存当前 P95 (**仅在样本 $\geq \text{minSamples}$ 时才覆盖**；不足则保留更早的 `lastFinalP95` 不动)。

Q5 决议 · 连续 reset 的 fallback 行为：

玩家 A：上局收摊时 P95=300，已存为 `lastFinalP95=300`。

切武器后样本积累到 15 (< 20) 时再切一次武器：第二次 reset 触发 `resetForPlayer`，`w.size()`=15 < 20，**不覆盖 lastFinalP95**，仍保持 300。

含义：连续切武器 / 关卡的瞬间，fallback 阈值始终基于"最后一次有充分样本的 P95"，避免被中间不足样本的窗口污染。

5. P95 计算规范

5.1 入样规则

事件类型	入样?	理由
普通归属成功的 L1~L8 伤害	✓	主流样本
致死单击	✗	高光不该污染日常分布

事件类型	入样?	理由
任一 G_* 过滤拦截	✗	异常事件
治疗 (L9_HEAL)	✗	不是攻击
<code>damage < lowDamageFloor</code>	✗	DoT 噪声
<code>victim Defence > 50</code>	✗	石头僵尸类高护甲怪：减伤后输出偏小，会拉低 P95。伤害仍计入玩家账户，仅不入样

5.2 维度

per-player——按 attacker UUID 分桶，每个玩家维护自己的 RollingWindow，互不干扰。

理由：

- 多玩家武器水平差距大时各自独立：A 用神器 (P95≈5000) 不污染 B 用菜刀 (P95≈200) 的 outlier 阈值
- 武器切换 reset 自然分裂：A 切武器只 reset 自己，B/C/D 不受影响
- 奶妈 / 物理玩家分工：两种伤害分布各有独立窗口
- 反例代价可控：归属失败的事件无法查 P95，由 G7a 物理地板兜底（详见下面"取舍"）

取舍 · 归属失败的事件：

事件归属失败 (L9_NONE) 时拿不到 attackerUuid，无法查 P95，outlier 判定失能。但该事件本来就走 L9_NONE 不进玩家账户，是否再额外打 outlier 标签只影响诊断精度。**G7a 物理地板（与归属无关）作为兜底**——10 万级别的纯机制刀仍能被识别为 `oversize` / `lethal-mechanism`。

5.3 窗口

滑动 N=100 + per-player reset 触发器：

```
serverConfig.p95windowSize (默认 100)
```

reset 触发器	时机	影响范围
玩家主手切换	<code>PlayerInventoryIndex</code> 5 tick 扫描时检测玩家 X 的主手 ITEM registry id 变化	仅 reset 玩家 X 自己的窗口
关卡切换	<code>StageBoundaryDispatcher.onStageEnter</code>	reset 所有玩家窗口
玩家加入服务器	<code>ServerPlayConnectionEvents.JOIN</code>	新建空窗口（无 reset 概念）

reset 触发器	时机	影响范围
玩家离开服务器	<code>ServerPlayConnectionEvents.DISCONNECT</code>	销毁窗口（节省内存）；可由 <code>p95EvictOnDisconnect</code> 关闭

武器切换检测细则：

- 仅看主手；副手 / 装备槽变化不触发
- 比较 `ItemStack.getItem()` 的 `Registries.ITEM.getId(...)`，**不看 NBT 词条**（铁剑→铁剑不重置；铁剑→钻石剑重置；剑→空手重置）
- 每个 CTT 玩家维护一个 `lastSeenMainHandId`，只在 id 变化的那一帧触发自己窗口的 reset

5.4 数据结构

```
class PerPlayerP95Registry:
    windows:          Map<UUID, Rollingwindow>          // 每个 CTT 玩家一份
    lastFinalP95:      Map<UUID, Integer>                // 跨 reset 衔接的 fallback 数值
    lastResetReason:   Map<UUID, ResetReason>            // WEAPON_SWITCH / STAGE_SWITCH /
SESSION_START
    lastResetMs:       Map<UUID, Long>                   // 用于 L 面板"32s 前"

    observe(attackerUuid, damage):
        windows.computeIfAbsent(attackerUuid, Rollingwindow::new).observe(damage)

    p95(attackerUuid):
        Rollingwindow w = windows.get(attackerUuid)
        if w == null or w.size() < minSamples: return -1
        return w.p95()

    resetForPlayer(attackerUuid, reason):
        Rollingwindow w = windows.get(attackerUuid)
        if w != null and w.size() >= minSamples:
            lastFinalP95.put(attackerUuid, w.p95()) // 仅在样本 ≥ 20 时覆盖
        if w != null: w.clear()
        lastResetReason.put(attackerUuid, reason)
        lastResetMs.put(attackerUuid, currentMs)

    resetAll(reason):
        for uuid in windows.keys: resetForPlayer(uuid, reason)

    evict(attackerUuid): // 玩家离开服务器
        windows.remove(attackerUuid)
        lastFinalP95.remove(attackerUuid)
        lastResetReason.remove(attackerUuid)
        lastResetMs.remove(attackerUuid)

class Rollingwindow:
    capacity = 100 // ServerConfig.p95windowSize
```

```

insertionOrder: Deque<Integer>          // 按插入顺序排列，用于挤出最旧
sortedArray:    List<Integer>           // 维持升序，用于 O(1) 取 P95

observe(damage):
    if size == capacity:
        oldest = insertionOrder.removeFirst()
        sortedArray.remove(binarySearch(oldest))
    insertionOrder.addLast(damage)
    sortedArray.insertSorted(damage)

p95():
    idx = ceil(size * 0.95) - 1          // 25 个样本 → idx=23 (第 24 位)
    return sortedArray[idx]

size(): return insertionOrder.size()

clear():
    insertionOrder.clear()
    sortedArray.clear()

```

性能：100 元素的 `ArrayList` `insertSorted` = $O(N)$ 拷贝 + $O(\log N)$ 二分定位；每 tick 顶多几次 `observe`，CPU 开销可忽略。**4 玩家队 $\approx$ 4 KB 内存。**

5.5 入样的具体顺序（关键）

事件经过 `G_low` / `G3` / `G4` / `G2` / `G7a` / `G6` 全部未命中 → 进 `PendingDamageBuffer`
 入队时立即记 `entry.wasLethalAtBuffer = (victim.RedHearts ≤ 0)` // Q3 简单版
 ↓ 等 3 tick
`flush()` 中：

- `attackerInfo = AttackerProbe.tryAttribute(victim, tick)` // 纯查询，不写统计
`attackerUuid = attackerInfo.uuid` (可能 `null`)
- 取 `entry.wasLethalAtBuffer` (入 `buffer` 那一帧的致死定格，复活也不退判)
- `outlier` 判定 (仅 `attackerUuid != null` 时)：


```

p95 = registry.p95(attackerUuid)          // 样本不足返回 -1
threshold = (p95 > 0 ? p95 × 50 : lastFinalP95(attackerUuid) × 1.5 × 50)
若 threshold > 0 且 damage > threshold:
    entry.wasLethalAtBuffer → L9_FILTER (lethal-mechanism), contributors 仍记
    否则                      → L9_FILTER (outlier), contributors 不记
            
```
- 调用 `AttackerProbe.recordFromDamageShower(victim, damage, tick, forceLayer)`
`forceLayer` 为上一步决定的 `L9_FILTER` 子标签或 `null`
- 入 P95 训练样本的判定 (仅在以下条件全部满足时)：
 - `attackerUuid != null`
 - `!entry.wasLethalAtBuffer` (致死单击不入样)
 - 不是被 `outlier` 拦截
 - `damage ≥ lowDamageFloor`
 - `victim.Defence ≤ 50`
 - `→ registry.observe(attackerUuid, damage)`

6. 配置项清单

完整配置项加到 `ServerConfig` (双端时 `ClientModConfig` 同样补一份对称项)。建议在 `ServerConfigScreen` 新增 "过滤器"Tab。

```
// === 总闸 ===
public boolean filterEnabled = true;

// === G_low: 低伤害噪声 ===
public int lowDamageFloor = 5;

// === G3: 固定值黑名单 (v6.5.2 已存在) ===
public boolean filterInitHpJumps = true;
public int[] initHpJumpValues = {1000, 9000, 10000, 100000};

// === G4: 可疑怪物名单 (v6.5.8 已存在) ===
public boolean filterSuspectVictims = true;
public String[] suspectVictims = {"幽匿骷髅", "幽匿僵尸"};
public int suspectVictimDamageThreshold = 800;

// === G2: 状态机边界 ===
public boolean filterStateBoundary = true;
public int sessionBoundaryGuardTicks = 5;
public String[] suspectVictimTags = {"Coffin", "Prop", "NPC", "TestDummy", "Debug"};

// === G7a: 物理地板 ===
public boolean filterPhysicalCeil = true;
public int physicalCeilMultiplier = 3;          // damage > MaxHP × 3 → FILTER

// === G6: 重放守卫 ===
public boolean filterDuplicateReplay = true;

// === G7b: 死亡锚点 + P95 Outlier ===
public boolean useDeathAnchorBuffer = true;
public int deathAnchorGraceTicks = 3;
public int deathAnchorOutlierMultiplier = 50;   // K
public int deathAnchorMinSamples = 20;

// === P95 训练 (per-player) ===
public int p95WindowSize = 100;
public int defenceExclusionThreshold = 50;       // victim.Defence > 50 不入样
public boolean fallbackToPreviousP95 = true;
public double fallbackP95Multiplier = 1.5;
public boolean p95EvictOnDisconnect = true;     // 玩家离开服务器时销毁窗口节省内存

// === G5: mass-wipe 守卫 (可选, v6.7.x) ===
public boolean filterMassWipe = true;
public int massWipeMinVictims = 3;
public double massWipeHpRatio = 0.95;
```

7. 击杀人头与 contributors 整合

7.1 核心承诺

L9_FILTER 只阻断"伤害账户", 不阻断 VictimDamageContributors。
这是机制斩杀仍能算人头的机制。

7.2 数据流改造

路径	当前行为	本设计行为
伤害事件 → PlayerDamageStats.add	命中过滤跳过	同一——不计入玩家伤害账户
伤害事件 → VictimDamageContributors.add	命中过滤跳过	改为: lethal-mechanism / oversize / outlier 三类仍写入 contributors; 其它 reason (init-hp-jump / suspect-victim / paused / low-noise / duplicate) 仍跳过
VictimTombstone 死亡判定	从 contributors 取 last hitter	不变
PlayerKillStats.kill	tombstone 触发	不变

7.3 哪些 reason 计 contributors

reason	计 contributors?	理由
low-noise	✗	1~4 点伤害确实是该玩家造成, 但量太小; 让 contributors 仅记录"实质参战"
init-hp-jump	✗	怪物初始化, 不是任何玩家造成
suspect-victim	✗	形态切换 set, 不是玩家造成
state-boundary (paused / session)	✗	状态切换噪声
suspect-tag (Coffin / Prop)	✗	不是合法目标
duplicate	✗	重放, 已经在前一帧记过
oversize (非致死的物理地板触发)	✗	伤害异常大, 归属链可信度低, 归到玩家头上有风险
lethal-mechanism (致死的物理地板 / outlier 触发)	✓	玩家可能确实参战过; 让击杀归属走兜底
outlier (非致死)	✗	同 oversize

reason	计 contributors?	理由
mass-wipe	✗	集体清场，归属不可信

简单记忆：只有 `lethal-mechanism` 一个 reason 计 contributors。其它要么不是玩家造成，要么归属不可信。

7.4 极端场景检验

场景	期望	数据流验证
玩家 A 打了小吃货 50 点，2 tick 后地图脚本 set RedHearts=-103950 致死	击杀给 A，伤害不计 103950	A 的 50 点已写入 contributors；机制斩杀的 103950 经 G7a 判定 lethal-mechanism 也写入 contributors (A)；tombstone 取 A 作 last hitter → A +1 击杀
玩家从未打过怪，纯机制清场	进未分类击杀	contributors 为空，tombstone 找不到 last hitter
玩家 A DoT 烧死小怪（最后一刀 1 点伤害）	击杀给 A	A 之前的着火攻击已在 contributors (>5 点)；最后 1 点 low-noise 不写入但不影响
玩家 A 暴击秒杀 BOSS (damage = MaxHP × 0.8 但致死)	击杀+伤害都给 A	没触发 G7a (< MaxHP × 3) → buffer → 致死 + damage ≤ P95 × 50 → 完整放行
玩家 A 暴击伤害爆表 + 致死 (damage = MaxHP × 5)	击杀给 A，伤害不计	G7a 触发 lethal-mechanism；contributors 仍记 A → tombstone 给 A 击杀；伤害不计入 PlayerDamageStats

第 5 行是与 R2 直接相关的判例：高光暴击如果真的爆到 MaxHP × 5，本设计会把伤害计入 L9_FILTER 而不是玩家账户。代价：极少数极端高光的伤害数字消失。**取舍接受**——这种伤害大概率是机制刀而不是合法暴击；玩家击杀仍计，K 表上"击杀+1"足以证明高光。

8. L9_FILTER reason 子标签扩展

8.1 reason 枚举

```
enum FilterReason {
    NONE,
    LOW_NOISE,           // damage < 5
    INIT_HP_JUMP,        // 黑名单值
    SUSPECT_VICTIM,      // 可疑怪物 + 阈值
    PAUSED,               // #PauseGame > 0
    SESSION_BOUNDARY,    // GameID 跳变 ≤ 5t
    SUSPECT_TAG,         // Coffin/Prop/NPC tag
    OVERSIZE,            // > MaxHP × 3 但未致死
    LETHAL_MECHANISM,    // > MaxHP × 3 且致死，或 > P95×K 且致死
    DUPLICATE,           // 重放
    OUTLIER,             // > P95 × K 且未致死
    MASS_WIPE,           // 同 tick ≥ N victim 全清场
}
```

```
}
```

8.2 聊天栏短标签

兼容 v6.5.2 的 [L9-FILT]，详情字段附 reason：

```
[L9-FILT:low-noise]      damage=2  victim=BlazingChicken
[L9-FILT:init-hp-jump]   damage=10000 victim=Husk
[L9-FILT:suspect-victim] damage=970   victim=幽匿骷髅
[L9-FILT:paused]         damage=42   reason=#PauseGame=1
[L9-FILT:session-boundary] damage=200  gameIdAge=2t
[L9-FILT:suspect-tag]    damage=8     tag=Coffin
[L9-FILT:oversize]       damage=3500  victim_maxHp=500 ratio=7.0x
[L9-FILT:lethal-mechanism] damage=103950 victim=小吃货 maxHp=500 → killer credit=Kirin0321
[L9-FILT:duplicate]      damage=120  prev_tick=1042
[L9-FILT:outlier]        damage=8800  p95=160 multiplier=55.0x
[L9-FILT:mass-wipe]      damage=2500  cohort_size=5
```

8.3 PlayerDamageStats 桶细分（可选）

`unattributedFiltered` 当前是单桶。建议增加 per-reason 计数到 `PlayerDamageStats` 或独立的 `FilterDiagReport`：

```
class FilterDiagReport:
    EnumMap<FilterReason, AtomicLong> eventCounts
    EnumMap<FilterReason, AtomicLong> damageSum

    observe(reason, damage):
        eventCounts[reason]++
        damageSum[reason] += damage
```

Q4 决议 · 不持久化： `FilterDiagReport` 仅在内存维护，不写入 NBT、不进 `playerstats.json`。session 重启 / 服务器重启即清零。

理由：诊断数据用于“本局看一眼调参”，跨局保留意义不大；省一份 NBT schema 演进负担。

玩家账户里的 `unattributedFiltered`（汇总桶）仍照常持久化，与 v6.5.x 行为一致。

8.4 L 键面板显示规范

8.4.1 常规视图顶部（默认开启）

L 键面板顶部加一行 **过滤器状态摘要**，与 `grandTotal` 同级，每局都看得见。显示当前查看面板者（即客户端玩家自己）的 P95：

```
SESSION · grandTotal=12,432  events=87  maxHit=2,310
◆ FILTER · 我的P95=320(87/100)  threshold=16,000  reset=武器切换32s前
```

字段含义：

字段	含义
我的P95=320	客户端玩家自己 (<code>MinecraftClient.getInstance().player.getUuid()</code>) 的 P95 值；样本 < minSamples 时显示 我的P95=--
(87/100)	当前样本数 / 窗口容量
threshold=16,000	当前 outlier 阈值 = 我的 P95 × K (即 320 × 50) 。样本不足时显示 fallback 阈值 (lastP95 × 1.5 × K) + * 标记
reset=...	最近一次 reset 触发器与时间。可能值：武器切换 / 关卡切换 / 局开始

样本不足时显示样式：

◆ FILTER · 我的P95=--(15/100) threshold=21,000* fallback reset=武器切换4s前

* 后缀表示当前阈值来自 lastFinalP95×1.5×K 的 fallback，不是真实 P95。

队友的 P95 看哪里：8.4.2 子页签里有完整的 per-player 表格。常规视图只显示自己以避免吵。

8.4.2 详情子页签（按一次 L 键切到）

L 键面板已经支持多视图切换 (v6.6.x 的 `DamagePanelRenderer`) 。新增 "过滤器诊断"子页签，按一次 L 切到：

过滤器诊断（本局累计）

low-noise:	427 events / 1,234 dmg	不入 P95
init-hp-jump:	27 events / 270,000 dmg	
suspect-victim:	14 events / 13,580 dmg	
paused:	3 events / 84 dmg	
session-boundary:	1 events / 200 dmg	
suspect-tag:	0 events	
oversize:	2 events / 10,500 dmg	
lethal-mechanism:	1 events / 103,950 dmg	← 击杀已计 +1
duplicate:	0 events	
outlier:	0 events	
mass-wipe:	0 events	

P95 训练窗口（per-player）

玩家	P95	样本	阈值	最近 reset
kirin0321	320	87/100	16,000	武器切换 32s 前
队友A	5,800	45/100	290,000	武器切换 12s 前
队友B	180	76/100	9,000	关卡切换 5m 前
队友C	--	18/100	21,000*	武器切换 4s 前 (fallback)

全局参数

K (outlierMultiplier):

50

minSamples:

20

物理地板倍数:

MaxHP × 3

低噪声地板:

< 5

Defence 排除阈值：> 50（伤害仍记账，但不入 P95）

字段说明：

字段	含义
P95	该玩家当前窗口的 P95；样本 < 20 时显示 —
样本	当前样本数 / 窗口容量
阈值	该玩家自己的 outlier 阈值；后缀 * 表示在用 fallback ($\text{lastFinalP95} \times 1.5 \times K$)
最近 reset	该玩家上次 reset 的触发器 + 时间

8.4.3 显示开关

```
ServerConfig.showFilterStatusInLPanel = true    // 8.4.1 常规视图顶部
ServerConfig.showFilterDiagSubPage = true      // 8.4.2 子页签
```

两个开关默认 ON。生产环境如果觉得吵可以单独关 8.4.1 留 8.4.2（dev mode 用）。

8.4.4 实时刷新频率

P95 数字每 tick 重算一次成本可观（排序数组查询 O(1)，但读 + 字符串拼接还是有开销）。建议：

- 8.4.1 常规视图：每秒（20 tick）刷一次缓存的 P95 字符串
- 8.4.2 子页签：打开时立即拉一次，后续每 0.5s（10 tick）刷一次

通过 `StatsSnapshotPayload v2` 把 `currentP95 / sampleCount / lastResetMs / filterDiagReport` 推到客户端缓存（v6.6.5 已经有 1Hz 推送框架）。

9. 验收用例

#	场景	输入	期望
T1	BOSS 9999 致死单击	victim=Boss, damage=9999, MaxHP=20000, RedHearts→-100	PASS（伤害+击杀都给玩家）
T2	AOE 一刀 5 怪	同 tick 5 victim 各 95% MaxHP，归属同一玩家	全 PASS（mass-wipe 因归属同一玩家不触发）
T3	<code>/kill @e[tag=CTTA11]</code>	同 tick 10 victim 100% MaxHP，归属各异	全 mass-wipe 过滤
T4	怪物初始化 set RedHearts=-10000	damage=10000	init-hp-jump 过滤

#	场景	输入	期望
T5	幽匿骷髅形态切换 970	victim 名含"幽匿骷髅", damage=970	suspect-victim 过滤
T6	#PauseGame=1 期间 任何伤害	damage=200	paused 过滤
T7	GameID 跳变后 5 tick 内	damage=300	session-boundary 过滤
T8	玩家高光致死 P95×30	victim 致死, damage=4800, P95=160	PASS (致死且 $\leq P95 \times 50$)
T9	非致死 outlier P95×60	victim 仍活, damage=9600, P95=160	outlier 过滤
T10	非致死 P95×3	victim 仍活, damage=480, P95=160	PASS
T11	BOSS 首战前 19 次 P95×20	samples=19, damage=3200	PASS (minSamples 未达)
T12	同 victim 同 damage 重复	tick=N 与 tick=N+1 完全相 同	第二条 duplicate 过滤
T13	DoT 烧伤 1 点	damage=1, victim 仍活	low-noise 过滤
T14	本次 issue 案例	victim=小吃货, MaxHP=500, damage=103950, 致死	lethal-mechanism 过滤; 击杀计给 Kirin0321 (last hitter, 2 tick 前刚打过 140 已在 contributors); 伤害账户不计 103950
T15	机制斩杀人头保留	玩家 A 打怪 50 点 (contributors 加 A), 2 tick 后机制 set 致死	A 击杀+1, A 伤害账户只 +50, 机制刀 103950 $\rightarrow$ L9_FILTER (lethal-mechanism)
T16	石头僵尸 Defence=70 受击 200	damage=200, Defence=70	PASS (伤害进玩家账户, 但不入 P95 训 练)
T17	武器切换瞬间敌方真 伤	玩家从铁剑切到钻石剑那一 tick 收到合法 200 伤害事件	PASS (reset 后样本不足 $\rightarrow$ fallback 到 lastP95 $\times 1.5 \times K$, 仍宽松)
T18	关卡切换瞬间	进新关 5 tick 内 damage=8000 victim=新关 BOSS	PASS (GameID 未跳? 或跳了 $\rightarrow$ session- boundary; 总之 P95 已 reset)
T19	dev mode 关闭过滤器	filterEnabled=false	所有事件直通归属链, 无任何过滤
T20	P95 fallback 边界	reset 后样本=15, lastP95=200, damage=14000	阈值=200 $\times 1.5 \times 50$ =15000, 14000 < 15000 $\rightarrow$ PASS

10. 分阶段落地

阶段	内容	工作量	价值
P1 · v6.7.0	G_low + G2 + G7a + G6	1~2 天	立刻解决物理地板可拦截的本次 case；状态机边界、低噪声、重放兜底
P2 · v6.7.1	PendingDamageBuffer 框架 + 死亡锚点（无 P95 outlier）	2~3 天	引入缓冲，把"致死分类"机制铺开
P3 · v6.7.2	RollingWindow + P95 计算 + minSamples 守卫 + Defence>50 排除 + fallbackToPreviousP95	2~3 天	P95 训练全套上线
P4 · v6.7.3	G7b outlier 子规则启用（K=50）+ 武器/关卡 reset 钩子	1~2 天	真正的 outlier 过滤
P5 · v6.7.4	G5 mass-wipe 守卫	1~2 天	处理批量清场
P6 · v6.7.5	UI / FilterDiagReport / L 面板诊断页	2 天	透明度 + 调参基础
P7 · v7.x	客户端等价（双端对称）	与 CLIENT_SIDE_STATS_PROPOSAL 同期	公服客户端 fallback

总工作量：单端约 9~14 天，可分多个版本独立交付。

10.1 每阶段独立可上线

每个 P 阶段都是**独立可发布的小版本**：

- P1 不依赖 P2~P7：v6.7.0 单独上线就能拦本次 case（物理地板）
- P2~P3 是 P4 的前置，但可以"P2 + P3 都上线但 P4 默认关闭"逐步放开
- P5 与 P4 完全独立，可以并行

10.1a P1 实装进度（持续追加）

子规则	上线版本	状态	备注
基础设施（FilterReason / FilterDecision / FilterDiagReport / DamageFilterPipeline）	v6.7.0	✅	DamageProbe.recordFromRedHearts 接入 applyFilters，filterEnabled 总闸

子规则	上线版本	状态	备注
G3 init-hp-jump 收编 (v6.5.2 旧规则)	v6.7.0	✓	DamageProbe.isInitHpJump 转发到 pipeline, <code>initHpJumpFilteredCount</code> 兼容旧 UI
G4 suspect-victim 收编 (v6.5.8 旧规则)	v6.7.0	✓	同上
G_low 低伤害噪声地板	v6.7.0	✓	<code>lowDamageFloor=5</code>
G_low Defence 豁免修复	v6.7.1	✓	实测发现 1~4 点合法伤害到石头僵尸被误过滤; <code>defenceExclusionThreshold=50</code> 提前到 P1, 详见 §4.2 / §5.1
G7a 物理地板	v6.7.2	✓	<code>damage > MaxHP × physicalCeilMultiplier</code> ; 致死 → <code>lethal-mechanism</code> , 非致死 → <code>oversize</code> 。P1 阶段 contributors 写入暂同 init-hp-jump (不写); 待 P3~P4 PendingDamageBuffer 上线后统一改为 <code>lethal-mechanism</code> 仍写 contributors
G2 状态机边界 (paused / session-boundary / suspect-tag)	TBD	⌚	待办
G6 重放守卫	TBD	⌚	待办 (命中率 < 0.1%, sanity check)

10.2 验收流程

每阶段上线后跑两局 CTT, 把 `FilterDiagReport` 数据贴到 PR, 确认:

- 命中数符合预期 (low-noise > 0、init-hp-jump = 0~少量、outlier 为 0 或个位数)
- 无玩家投诉"我的伤害消失了"
- L 面板对比"现在的 P95"与历史几局, 节奏一致

11. 与失败教训的对照

失败教训	v6.3.8 / v6.3.10	本设计
误伤致死高光单击	均值×K=5 直接拦	R1 致死守门 + 物理地板物理上限 MaxHP×3: 致死 + 数值合理 → 必放行
BOSS 首战无样本 → 退化 hard cap	阈值未启用, 固定上限误伤	minSamples=20 + fallback 上一关 P95×1.5 + 物理地板三层兜底
武器升级后 P95 滞后	没有 reset 机制 (且全局共享)	武器切换 reset (per-player) + 关卡切换 reset (全员)
多玩家武器水平差距大互相污染	per-player 但归属错乱	per-player 维度 + tryAttribute 纯查询接口

失败教训	v6.3.8 / v6.3.10	本设计
均值被一次假伤害带飞	均值算法	改用 P95，对 outlier 极不敏感
一刀切回滚	整套过滤一个开关	每条规则独立 boolean 开关 + 数值参数可配
玩家不知道为什么被过滤	仅 log 输出	L9_FILTER:reason 子标签 + L 面板诊断页 + 聊天栏可见

12. 客户端等价 (v7 fallback)

CLIENT_SIDE_STATS_PROPOSAL.md §4.7 应增加本节内容，描述客户端版的等价过滤：

12.1 完全对等的规则

- G_low, G3, G4, G7a (物理地板需读 victim MaxHP 计分板，客户端可见)
- G6 (duplicate 守卫)
- G7b (PendingDamageBuffer + P95 训练)
- 武器切换 reset: 用 ClientPlayerEntity 主手 ItemStack 监听
- 关卡切换 reset: 复用 ClientStageProbe.onStageEnter

12.2 需要等价替代的规则

服务端规则	客户端替代
G2 #PauseGame > 0	客户端读 #PauseGame CTT scoreboard，等价
G2 Coffin / Prop / NPC tag	客户端拿不到 commandTags (不跨网络同步)，改用启发式：MaxHP > 0 但 RedHearts ≤ 0 持续 ≥ 5 tick 仍存活 → 视为 Coffin
wasLethalAtBuffer 致死定格	客户端入 buffer 时同样读一次 victim.RedHearts CTT 计分板值，等价 (无需依赖死亡 race)
Defence > 50 排除	客户端读 victim Defence 计分板，等价

12.3 精度差异

客户端版过滤命中率预计与服务端**等价 ±2%**——所有过滤规则的输入数据 (计分板 score、4 层心数 delta、实体 UUID、MaxHP、Defence、RedHearts) 都是 vanilla S2C 同步的。唯一差异：

- Coffin tag 替代为启发式 → 偶尔误判 Coffin 为合法 victim
- 击杀人头归属在客户端不可达 (无 onDeath hook，仅靠实体消失 + contributors 后验)，但**致死过滤**通过 wasLethalAtBuffer 已对齐

总体精度足够支撑 fallback 模式的统计需求。

13. 与现有架构的整合点

现有点	改动
DamageProbe.recordFromRedHearts	G_low / G2 / G3 / G4 / G7a / G6 串成 applyFilters(...); 未命中送入 PendingDamageBuffer, 入队时记 wasLethalAtBuffer = (victim.RedHearts ≤ 0)
DamageProbe.recordFromDamageShower (主源 = DamageShower 时)	同上
DamageProbe.flushTick	末尾追加 victimTombstone.scanDeaths() → PendingDamageBuffer.flush(tickCounter) → MasswipeBucket.flush() (顺序固定: 先扫死亡再 flush, 保证 contributors → kill 归属链不丢)
AttackerProbe.recordFromDamageShower(..., forceLayer)	API 不变, 仅被 buffer 延迟调用
PlayerDamageStats.unattributedFiltered	不变; 新增 filterReasonCounts 与 filterReasonDamage map (仅内存, 不持久化, 见 §8.3)
VictimDamageContributors.add	新逻辑: reason == LETHAL_MECHANISM 时仍写入; 其它过滤 reason 跳过
AttackerProbe.tryAttribute(victim, tick) → AttackerInfo	新增: 与 attribute() 同样的归属逻辑, 但不写入任何统计, 仅返回 (uuid, layer) 二元组。供 PendingDamageBuffer.flush 在判 outlier 前查用
PerPlayerP95Registry	新增类: 见 §5.4
StageBoundaryDispatcher	新增 lastGameIdChangeTick() getter; onStageEnter 触发 PerPlayerP95Registry.resetAll(STAGE_SWITCH)
PlayerInventoryIndex	新增 per-player "主手 registry id 变更" 监听; 变化时触发 PerPlayerP95Registry.resetForPlayer(uuid, WEAPON_SWITCH) (连续 reset 时若新窗口样本 < minSamples, 不覆盖 lastFinalP95, 见 §4.8.3)
ServerPlayConnectionEvents.JOIN / DISCONNECT	注册 hook: JOIN 无操作 (首次 observe 时懒初始化); DISCONNECT 调 registry.evict(uuid)
VictimTombstone	不需 diedBetween 方法 (Q3 简单版改用 wasLethalAtBuffer flag); 保留 scanDeaths() 用于击杀归属链路
ScoreboardUpdateMixin	不动——所有过滤都在 DamageProbe 之内
ServerConfigScreen	新增"过滤器"Tab, 展示开关 + 实时计数 (数据来源 FilterDiagReport, 仅内存)

Q2 决议 · tick 末尾事件顺序固定: VictimTombstone.scanDeaths → PendingDamageBuffer.flush → MasswipeBucket.flush。

注意 Q3 简单版下 flush 不再读 VictimTombstone 判致死, 但 scanDeaths 仍要先跑——因为 contributors 触发的"击杀归属"事件需要在本 tick 未完成, 否则 [L9-FILT:lethal-mechanism] 日志中的 "→ killer credit=Kirin0321" 字段会延后一 tick。

维护说明

- 各 Phase 完成后请在 §10 勾选并加 commit 引用
- 新发现的异常源类型应回填到 §2 表格
- 实测命中数据填入 §11 与 §9 用例

- 若新增 reason 子类型，同步更新 §8.1 枚举与 §7.3 表格

修订历史

日期	版本	改动
2026-04-27	v1.0	初稿，5 个用户决策拍板：全局一个 P95、N=100、主手 registry id reset、Defence>50 排除入样、fallback 上一 P95×1.5
2026-04-27	v1.1	维度从全局改 per-player ：每个 attacker UUID 一个 RollingWindow；reset 触发器自然分裂；新增 PerPlayerP95Registry 数据结构与 AttackerProbe.tryAttribute 纯查询接口；L 面板常规视图显示自己的 P95，子页签表格显示队友 P95。代价：归属失败的事件 outlier 失能，由 G7a 物理地板兜底
2026-04-27	v1.2	Q2~Q5 决议拍板，可进入 P1 实施 ：① Q2 tick 末顺序固定 scanDeaths → buffer.flush → masswipe.flush；② Q3 致死判定改简单版 wasLethalAtBuffer（入 buffer 那帧 RedHearts ≤ 0 即定格，复活也不退判，不依赖 VictimTombstone race）；③ Q4 FilterDiagReport 仅内存维护，不持久化 NBT ，session 重启清零；④ Q5 连续 reset 时若新窗口样本 < minSamples 则 不覆盖 lastFinalP95，保持上一份有效 P95×1.5×K 作 fallback。客户端版同步对齐 wasLethalAtBuffer
2026-04-27	v1.3	G_low 加 Defence 豁免 （实测发现误伤）：石头僵尸 / 远古卫士这类高护甲怪被普通武器打减伤后正常输出 1~4 点，被 G_low 一刀切误过滤。 修复 ：G_low 判定前先读 victim.Defence（地图自定义计分板），> defenceExclusionThreshold（默认 50）时跳过 G_low——伤害正常进玩家账户。defenceExclusionThreshold 配置项 从 P3 提前引入到 P1 ，与 §5.1 P95 入样规则共用同一阈值。地图无 Defence scoreboard 时降级到原 G_low 行为，不破坏兼容性
2026-04-27	v1.4	G7a 物理地板落地 （P1 第三块拼图）：新增 ServerConfig.filterPhysicalCeil + physicalCeilMultiplier（默认 3）；DamageFilterPipeline.applyFilters 在 G_low / G3 / G4 之后插入 G7a 判定——damage > victim.MaxHP × multiplier 时，致死（isRemoved 或 RedHearts ≤ 0）打 lethal-mechanism，非致死打 oversize。新增 readVictimMaxHp / readVictimRedHearts 工具方法，与 readVictimDefence 合并到通用 readVictimScoreboard(victim, objName)。P1 阶段 contributors 写入策略 保持原状 （同 init-hp-jump 不写）—— [L9-FILT:lethal-mechanism] 的击杀归属暂时仍走 victimLethalCandidate 兜底链路；待 P3~P4 引入 PendingDamageBuffer 后，统一在 buffer.flush 改 AttackerProbe 接受 reason，让 LETHAL_MECHANISM / OVERSIZE / OUTLIER 仍写 contributors（详见 §13 表格"VictimDamageContributors.add"行）。本版起 §10.1a 跟踪 P1 实装进度